

BAIT AL MANDI

System Architecture Documentation

بيت المندي — وثيقة البنية المعمارية للنظام

Version 1.0

June 15, 2026

Name:	Bait Al Mandi Restaurant
Project:	Full-Stack Restaurant Management System
Tech Stack:	Next.js 14 · Supabase · TypeScript
Version:	1.0
Created:	June 15, 2026
Exported:	June 15, 2026
Classification:	Confidential

Table of Contents

Section 1: System Overview

1.1 What is Bait Al Mandi?

1.2 Problem & Solution

1.3 Tech Stack

1.4 Key Versions

Section 2: Directory Structure

Section 3: Database Architecture

3.1 Core Tables (22 Tables)

3.2 Enums (7)

3.3 Orders Table Structure

3.4 Site Settings (Key-Value Store)

Section 4: Pages

4.1 Public Pages

4.2 Admin Pages

Section 5: API Routes

5.1 Public Routes

5.2 Report Routes

Section 6: Order Lifecycle

Section 7: Delivery System

7.1 Delivery Fee Calculation Flow

7.2 Pricing Formula

7.3 OSRM Integration Flow

Section 8: Offer & Bundle System

8.1 Offer Types (4)

8.2 Pricing Engine (offer-pricing.ts)

8.3 Snapshots

Section 9: Invoice System

9.1 Two Components, One Source

9.2 Invoice Content Layout

9.3 PDF Specifications

9.4 Print Specifications (Admin)

9.5 Tracking Token System

Section 10: Security & Authorization

10.1 Roles (3)

10.2 Access Control

10.3 Server Action Protection

10.4 Potential Vulnerabilities

Section 11: Reporting System

11.1 Architecture

11.2 Print & Export

Section 12: State Management (Stores)

12.1 cartStore (Zustand + persist)

12.2 SettingsContext (React Context)

Section 13: Legacy Files (Unused)

Section 14: Dependency Map

.....

Section 15: Executive Summary

.....

15.1 Strengths

.....

15.2 Recommended Improvements

.....

15.3 Project Deliverables

.....

Section 1: System Overview

1.1 What is Bait Al Mandi?

A full-stack restaurant management system for **Bait Al Mandi** — a Yemeni restaurant in **Sanaa, Yemen**. The system provides:

- **Public Website** (Landing Page, Menu, Cart, Ordering, Tracking)
- **Admin Dashboard** (Dashboard, Orders, Menu CRUD, Categories, Offers, Gallery, Reviews, Reports, Delivery Settings, Site Settings)
- **Delivery System** (Distance-based pricing, weather/peak surcharges, OSRM Routing, interactive map)
- **Invoice System** (PNG, PDF, direct print, QR Code for tracking)
- **Offer & Bundle System** (Pricing Engine: fixed price, percentage discount, amount discount, free item)
- **Order Tracking System** (Unique Tracking Token per customer, QR Code)
- **Reports & Analytics** (12 API routes + interface tabs + print + Excel)

1.2 Problem & Solution

Problem	Solution
No digital restaurant management system	Complete system from ordering to delivery to reports
Delivery without maps or fair pricing	OSRM Routing + distance-based pricing + weather/peak surcharges
Traditional paper invoices	PNG/PDF/print invoices with QR tracking
Complex offers without a pricing engine	Pricing Engine with 4 offer types
Order tracking difficulty	Tracking Token + tracking page per customer

1.3 Tech Stack

- **Framework:** Next.js 14.2.3 (App Router)
- **Language:** TypeScript 5.4.5
- **Styling:** Tailwind CSS + CSS Modules
- **Database:** Supabase (PostgreSQL 15)
- **ORM:** Drizzle ORM + Supabase REST (dual access)
- **Auth:** Supabase Auth (email/password) + Row Level Security
- **State (Client):** Zustand (cartStore) + React Context (Settings)
- **Charts:** Recharts
- **Maps:** Leaflet + OSRM (routing)
- **GIS:** Turf.js + RBush (R-Tree spatial indexing)
- **PDF/PNG:** jsPDF + html2canvas + QRCode.react
- **Excel:** ExcelJS
- **Animations:** Framer Motion + GSAP
- **Email:** Nodemailer

1.4 Key Versions

Package	Version
Next.js	^14.2.3
React	^18.3.1
Supabase SSR	^0.5.1
Drizzle ORM	^0.30.10
Zustand	^5.0.0

Section 2: Directory Structure

```

baitalmandiwibapp/
├─ src/
│   ├─ actions/          # Server Actions (RSC mutations)
│   │   ├─ categories.ts  # Categories CRUD
│   │   ├─ items.ts       # Items & Prices CRUD
│   │   ├─ orders.ts      # Order creation + delivery calc + status update
│   │   └─ orders-offers.ts # Fetch offer info for order
│   │
│   └─ app/              # App Router (pages + API)
│       ├─ page.tsx       # Landing Page
│       ├─ layout.tsx     # Root Layout (Navbar, Footer, SettingsProvider)
│       ├─ globals.css    # Global styles (Custom Properties)
│       ├─ page.module.css # CSS Module for landing page
│       ├─ favicon.ico
│       ├─ menu/page.tsx  # Menu page (view + search + cart)
│       ├─ cart/page.tsx  # Cart page (legacy)
│       ├─ my-orders/page.tsx # Cart + map + full order flow
│       ├─ contact/page.tsx # Contact page + review submission
│       ├─ gallery/page.tsx # Photo gallery
│       ├─ test-map/page.tsx # Map test page
│       ├─ t/[token]/page.tsx # Order tracking by token
│       ├─ track-order/[orderId]/page.tsx # Order tracking by UUID
│       ├─ admin/         # Admin panel
│       │   ├─ layout.tsx # Admin Layout (Sidebar + Auth)
│       │   ├─ page.tsx   # Admin Dashboard
│       │   ├─ login/     # Login page + actions
│       │   ├─ orders/    # Order management
│       │   ├─ menu/      # Menu CRUD
│       │   ├─ categories/ # Categories CRUD
│       │   ├─ offers/    # Offers CRUD
│       │   ├─ gallery/   # Gallery management
│       │   ├─ reviews/   # Reviews management
│       │   ├─ reports/    # Reports (9 tabs)
│       │   ├─ delivery/   # Delivery settings
│       │   └─ settings/   # Site settings
│       └─ api/           # API Routes
│           ├─ auth/login/route.ts
│           ├─ resolve-zone/route.ts (legacy)
│           └─ reports/ (12 endpoints)
│
├─ components/
│   ├─ admin/reports/    # 9 report tab components
│   ├─ invoice/
│   │   ├─ InvoiceModal.tsx # Invoice modal (PNG/PDF)
│   │   └─ receipt-html.ts  # Receipt HTML for printing
│   └─ layout/           # Navbar, Footer, LoadingScreen
│
└─ db/
    ├─ index.ts          # Drizzle ORM initialization
    └─ schema.ts          # Complete schema (22 tables)

```



```

└─ rls.sql          # Row Level Security policies
└─ lib/
  └─ supabase.ts     # Supabase Client (Browser)
  └─ permissions.ts  # Permission system (3 roles)
  └─ constants.ts    # Constants (contact info)
  └─ settings-context.tsx # React Context for settings
  └─ delivery-pricing.ts # Single source for delivery fee calc
  └─ delivery-routing.ts # OSRM Routing + fallback
  └─ delivery-zones.ts # (Legacy) zone system
  └─ offer-pricing.ts # Offer pricing engine
  └─ bundle-utils.ts  # Bundle info extraction
  └─ location-validation.ts # Sanaa location validation
  └─ exportExcel.ts   # Excel export
  └─ printReport.ts   # Report printing
  └─ geo/             # (Legacy) zone resolution
└─ middleware.ts     # Next.js Middleware (Admin Routes)
└─ types.d.ts        # Global TypeScript types
└─ services/reports/ # Report engine (Drizzle-based)
└─ utils/supabase/   # Supabase utilities
└─ supabase/migrations/ # 11 SQL Migrations (0000-0010)
└─ sanaa-map-integration/ # GeoJSON data for Sanaa
└─ drizzle.config.ts
└─ next.config.js
└─ tailwind.config.ts
└─ tsconfig.json
└─ package.json
└─ .env.local

```

Section 3: Database Architecture

3.1 Core Tables (22 Tables)

#	Table	Description	Key Fields
1	users	Restaurant customers	id (PK), full_name , phone (UK)
2	admin_users	Admin accounts	id (PK), email (UK), role (enum)
3	categories	Menu categories	id (PK), slug (UK), sort_order
4	items	Menu items	id (PK), category_id (FK)
5	item_prices	Item price variants	id (PK), item_id (FK), original_price
6	offers	Discount offers/bundles	id (PK), offer_type , discount_percent
7	offer_items	Bundle components	id (PK), offer_id (FK), menu_item_id (FK)
8	cart_sessions	Cart sessions (legacy)	id (PK), session_id (UK)
9	cart_items	Cart line items (legacy)	id (PK), session_id (FK)
10	order_sequences	Order number sequences	id (PK), sequence_date (UK)
11	orders	Core table	id (PK), order_number (UK), 28 columns
12	order_items	Order line items	id (PK), order_id (FK)
13	order_status_history	Status change audit trail	id (PK), order_id (FK)
14	order_offers	Offer snapshot at order	id (PK), order_id (FK, CASCADE)
15	order_offer_items	Bundle lines snapshot	id (PK), order_offer_id (FK, CASCADE)
16	gallery_images	Photo gallery	id (PK), image_url

17	<code>reviews</code>	Customer reviews	<code>id</code> (PK), <code>rating</code>
18	<code>site_settings</code>	Key-value settings store	<code>id</code> (PK), <code>setting_key</code> (UK), <code>value</code> (jsonb)
19	<code>branches</code>	Restaurant branches	<code>id</code> (PK), <code>name_ar/en</code>
20	<code>audit_logs</code>	Admin audit logs	<code>id</code> (PK), <code>entity_id</code>
21	<code>customer_tokens</code>	Phone to token mapping	<code>phone</code> (PK), <code>tracking_token</code>
22	<code>scheduled_reports</code>	Scheduled email reports	<code>id</code> (PK), <code>period</code> (enum)

3.2 Enums (7)

```

admin_role      → 'developer' | 'manager' | 'order_manager'
offer_status    → 'active' | 'expired' | 'disabled'
order_method    → 'whatsapp' | 'website'
order_status    → 'pending' | 'confirmed' | 'preparing' | 'on_the_way' |
'delivered' | 'cancelled'
payment_method  → 'cash' | 'transfer' | 'wallet'
audit_action    → 'modify' | 'cancel' | 'status_change' | 'other'
report_period   → 'daily' | 'weekly' | 'monthly'

```

3.3 Orders Table Structure

```

orders (
  id                UUID PK DEFAULT gen_random_uuid()
  order_number      TEXT UK          ← "BAM-YYYYMMDD-XXXX"
  customer_id       UUID FK→users
  customer_name     TEXT NOT NULL
  customer_phone    TEXT NOT NULL
  tracking_token    TEXT              ← UUID for tracking
  delivery_address  TEXT NOT NULL    ← Required (≥10 chars)
  subtotal         NUMERIC NOT NULL ← Server-calculated
  delivery_fee      NUMERIC DEFAULT 0
  tax_amount       NUMERIC DEFAULT 0
  total_amount     NUMERIC NOT NULL ← Server recalculation
  notes            TEXT
  estimated_time    TIMESTAMPTZ
  order_method     order_method NOT NULL
  payment_method   payment_method DEFAULT 'cash'
  offer_id         UUID FK→offers
  status           order_status DEFAULT 'pending'
  version          INTEGER DEFAULT 1 ← Optimistic Locking
  created_at / updated_at TIMESTAMPTZ
  is_archived / archived_at BOOLEAN + TIMESTAMPTZ
  is_deleted / deleted_at  BOOLEAN + TIMESTAMPTZ

  -- Delivery Geo Fields
  delivery_latitude    NUMERIC
  delivery_longitude   NUMERIC
  delivery_zone        TEXT          ← null (resolved)
  delivery_distance_km NUMERIC
  delivery_duration_minutes INTEGER
  location_verified    BOOLEAN DEFAULT false

  -- Delivery Fee Snapshot Fields (migration 0010)
  base_delivery_fee_amount NUMERIC DEFAULT 0
  extra_distance_km        NUMERIC DEFAULT 0
  extra_fee_amount         NUMERIC DEFAULT 0
  weather_fee_amount       NUMERIC DEFAULT 0
  peak_fee_amount          NUMERIC DEFAULT 0
  peak_percentage_used     NUMERIC DEFAULT 0
)

```

3.4 Site Settings (Key-Value Store)

The `site_settings` table stores all configurable parameters:

Key	Type	Default	Description
restaurant_name	string	"بيت المندي"	Restaurant name
restaurant_image	string	""	Restaurant logo URL
restaurant_lat	string	"15.360..."	Restaurant latitude
restaurant_lng	string	"44.174..."	Restaurant longitude
base_delivery_fee	string	"400"	Base delivery fee
included_distance_km	string	"2"	Included distance (km)
extra_fee_per_km	string	"100"	Extra fee per km
max_delivery_distance_km	string	"15"	Max delivery distance
road_factor	string	"1.5"	Road factor (Haversine→Road)
enable_weather_fee	string	"false"	Enable weather surcharge
weather_fee	string	"200"	Weather fee amount
enable_peak_hours	string	"false"	Enable peak hour surcharge
peak_start_time	string	"12:00"	Peak start time
peak_end_time	string	"14:00"	Peak end time
peak_percentage	string	"20"	Peak surcharge percentage
peak_days	string	"[]"	Peak days (JSON array)

Section 4: Pages

4.1 Public Pages

Route	Component	Type	Function
/	page.tsx	Client	Landing Page: Hero, Menu, Stats, Reviews, Gallery, WhatsApp
/menu	page.tsx	Client	Full menu with search, categories, cart, offers
/my-orders	page.tsx	Client	Cart + map + delivery calc + order creation
/cart	page.tsx	Client	Legacy cart page
/gallery	page.tsx	Client	Photo gallery with Lightbox
/contact	page.tsx	Client	Contact form + review + restaurant info
/t/[token]	page.tsx	Client	Order tracking by token with InvoiceModal
/track-order/[orderId]	page.tsx	Client	Legacy tracking by UUID
/test-map	page.tsx	Client	Map and OSRM test page

4.2 Admin Pages

Route	Component	Function
/admin	page.tsx	Dashboard: totals, revenue, active items
/admin/login	page.tsx	Admin login (Supabase Auth)
/admin/orders	page.tsx	Order management (status updates, details)
/admin/menu	page.tsx	Menu items CRUD with images and prices
/admin/categories	page.tsx	Categories CRUD
/admin/offers	page.tsx	Offers CRUD (4 pricing types)
/admin/gallery	page.tsx	Gallery image management
/admin/reviews	page.tsx	Review management (approve/reject)
/admin/reports	page.tsx	Reports (9 tabs with print/export)
/admin/delivery	page.tsx	Delivery settings (distance, weather, peak)
/admin/settings	page.tsx	Site settings (restaurant, contact, bank)

Section 5: API Routes

5.1 Public Routes

Route	Method	Function
/api/auth/login	POST	Admin login (Supabase Auth + Cookies)
/api/resolve-zone	POST	Legacy zone resolution from coordinates

5.2 Report Routes (All GET)

Route	Function
/api/reports/dashboard	Today's sales, order count, last 10 orders
/api/reports/orders	Order logs with filtering (status, payment, search)
/api/reports/products	Product analysis (top 10, categories, unsold)
/api/reports/customers	Customer analysis (top 50)
/api/reports/compare	Period comparison (sales growth, orders, distribution)
/api/reports/offers	Offer analysis (total discounts, top 10)
/api/reports/invoices	Invoice listing
/api/reports/audit	Status change audit log
/api/reports/sales	Sales analytics (Drizzle ORM)
/api/reports/delivery-analytics	Delivery analytics (fees by day/distance)
/api/reports/cron	Protected cron endpoint for scheduled reports
/api/reports/schedule	POST: Schedule email reports

Section 6: Order Lifecycle

COMPLETE ORDER LIFECYCLE

1. Browse Menu

- └ Client: /menu → display items with prices
- └ Client: useCartStore (Zustand, localStorage "bam-cart-storage")
- └ Support for bundles and individual offers

2. Create Order (/my-orders)

- └ Client: Enter name, phone, address (required ≥10 chars)
- └ Client: Select location on map (Leaflet)
- └ Client: calculateDeliveryFeeServer(lat, lng) ← Server Action
 - └ Verify Sanaa location (isInsideSanaa)
 - └ Calculate OSRM distance (calculateRoute)
 - └ Calculate delivery fee (calculateDeliveryFee)
- └ Client: Submit order (createOrder)
- └ Server Action: createOrder()
 - └ 1. Validate input data
 - └ 2. Fetch delivery settings (fetchDeliverySettings)
 - └ 3. Verify location (isInsideSanaa + max distance)
 - └ 4. Calculate distance (OSRM → Haversine fallback)
 - └ 5. Calculate delivery fee (calculateDeliveryFee)
 - └ 6. Generate order number (generateOrderNumber with Optimistic Locking)
 - └ 7. Manage tracking token (get/create from customer_tokens)
 - └ 8. Validate offer (if applicable: offer validation + pricing)
 - └ 9. Save order (orders table + order_items)
 - └ 10. Save initial status (order_status_history)
 - └ 11. Save offer snapshot (order_offers + order_offer_items)
 - └ 12. Return { order, trackingToken }

3. Track Order

- └ Customer: /t/[token] → tracking page with QR + InvoiceModal
- └ Admin: /admin/orders → status management

4. Update Status (Admin)

- └ updateOrderStatus(orderId, newStatus, adminId?)
 - └ Read current status + version
 - └ Update with Optimistic Locking (.eq('version', currentVersion))
 - └ Log in order_status_history
 - └ Handle concurrent modification conflicts

5. Invoice

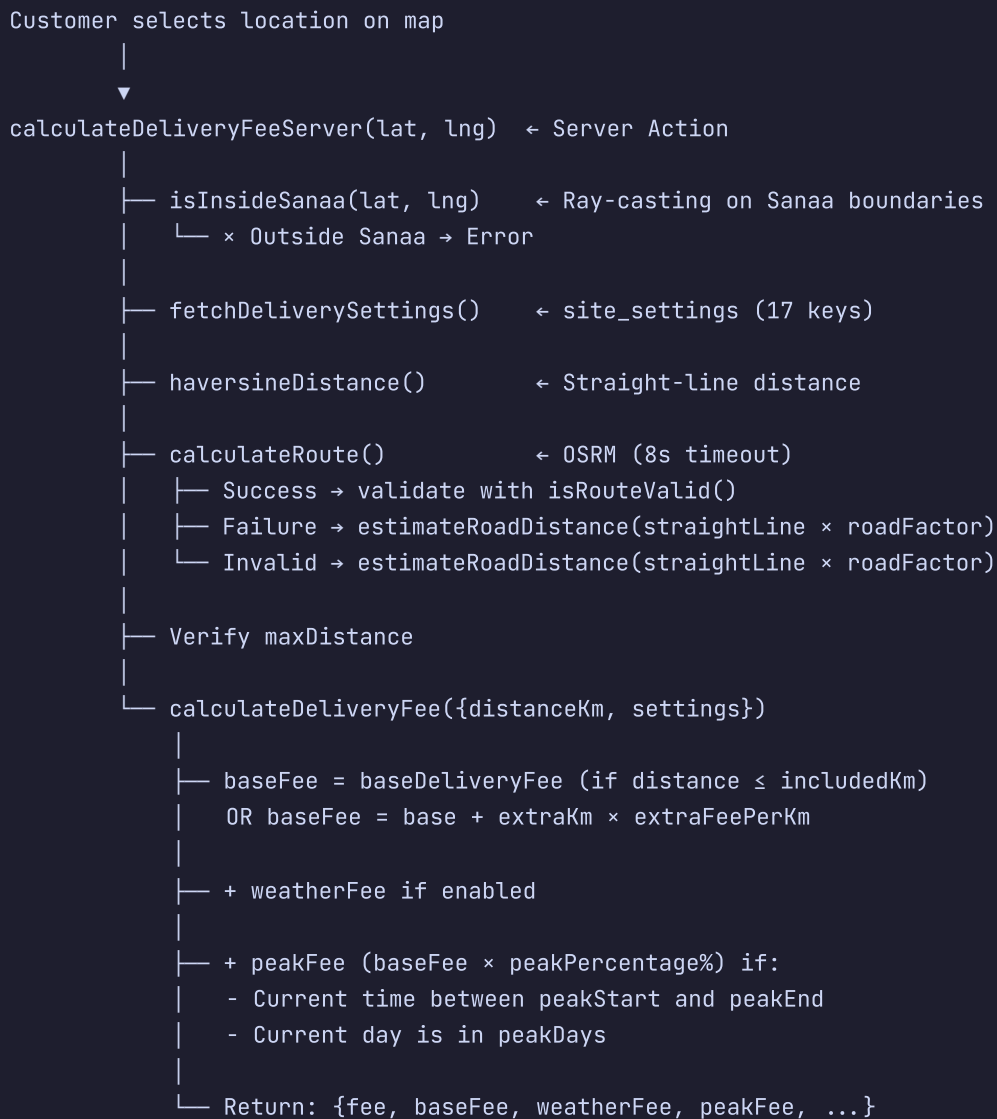
- └ Client: InvoiceModal → PNG (html2canvas) / PDF (jsPDF)
- └ Admin: receipt-html.ts → direct print (window.print)

6. Archive

- └ is_archived = true / is_deleted = true (soft delete)

Section 7: Delivery System

7.1 Delivery Fee Calculation Flow



7.2 Pricing Formula

```

If distanceKm ≤ includedDistanceKm:
    deliveryFee = baseDeliveryFee
Otherwise:
    deliveryFee = baseDeliveryFee + (distanceKm - includedDistanceKm) ×
    extraFeePerKm

Additional surcharges (if applicable):
    + weatherFee (if enabled)
    + baseFee × peakPercentage / 100 (if peak time and peak day)

Default values:
    baseDeliveryFee = 400 YER
    includedDistanceKm = 2 km
    extraFeePerKm = 100 YER/km
    maxDeliveryDistanceKm = 15 km
    weatherFee = 200 YER
    peakPercentage = 20%

```

7.3 OSRM Integration Flow

```

calculateRoute(originLat, originLng, destLat, destLng, straightLineKm,
roadFactor)
|
|— URL: https://router.project-osrm.org/route/v1/driving/{lng},{lat};
{lng},{lat}
|
|— Response: distance (meters), duration (seconds)
|
|— isRouteValid(osrmKm, straightLineKm, roadFactor)
|   |— osrmKm < straightLine → false
|   |— osrmKm > straightLine × 10 → false
|   |— osrmKm < straightLine × 0.75 → false
|
|— Fallback: straightLineKm × roadFactor (default 1.5)

```

Section 8: Offer & Bundle System

8.1 Offer Types (4)

Type	Description	Example
<code>fixed_price</code>	Fixed price for bundle	"Family meal for 5000"
<code>percentage_discount</code>	Percentage discount	"20% off bundle"
<code>amount_discount</code>	Fixed amount discount	"1000 YER off"
<code>free_item</code>	Cheapest item free	"Buy 2 get 3rd free"

8.2 Pricing Engine (offer-pricing.ts)

```
calculateOfferPrice({ offerType, salePrice, discountPercent, discountAmount,
items })
  |
  |— originalPrice = Σ(item.unitPrice × item.quantity)
  |
  |— fixed_price      → finalPrice = salePrice ?? originalPrice
  |— percentage_discount → finalPrice = originalPrice × (1 -
discountPercent/100)
  |— amount_discount  → finalPrice = max(0, originalPrice -
discountAmount)
  |— free_item        → finalPrice = originalPrice -
unitPriceCheapestItem
  |
  |— return { originalPrice, discountAmount, discountPercent, finalPrice,
savings }
```

8.3 Snapshots

When creating an order with an offer:

1. `order_offers` : Stores `offer_name`, `original_price`, `discount_amount`, `final_price`
2. `order_offer_items` : Stores detailed bundle component data for each order

3. This ensures report accuracy even if the offer changes later

Section 9: Invoice System

9.1 Two Components, One Source

```
receipt-html.ts (Server/Print)
├─ generateReceiptBody(order, settings, trackUrl, bundleInfo)
│   └─ HTML string → used in:
│       ├─ admin/reports/InvoicesTab.tsx (direct print)
│       └─ InvoiceModal.tsx (design matching)
└─ generateReceiptHtml(order, settings, trackUrl, bundleInfo)
    └─ Complete HTML document for printing
        └─ Waits for images to load before window.print()

InvoiceModal.tsx (Client/Modal)
├─ Display invoice in modal (React component)
├─ Export PNG → html2canvas (scale: 3)
└─ Export PDF → jsPDF
    └─ format: [80mm, proportionalHeight] ← single page
        └─ addImage(imgData, 'PNG', 0, 0, 80mm, height)
```

9.2 Invoice Content Layout

```

| — Customer Invoice — |
| ----- |
| [Restaurant Logo] |
| Bait Al Mandi |
| Address |
| Phone: XXX - WhatsApp: XXX |
| ----- |
| Invoice # | BAM-XXXX.. |
| Date | June 15 |
| Time | 08:30 PM |
| Payment | Cash |
| ----- |
| Customer Info |
| Name: Mohammed |
| Phone: 777... |
| Address: ... |
| ----- |
| Order Details |
| Item | Size | Qty | Price | Tot |
| Mandi | Med | 2 | 1500 | 3000 |
| ----- |
| [Bundle Info if present] |
| ----- |
| Subtotal | 3,000 |
| Delivery Fee | 400 |
| ----- |
| Grand Total | 3,400 YER |
| ----- |
| [QR Code for tracking] |
| ----- |
| Thank you for your trust |
| Bait Al Mandi |
| © 2026 Bait Al Mandi |
| ----- |

```

9.3 PDF Specifications

- **Width:** 80mm (standard thermal receipt)
- **Height:** Proportional to canvas (single page only)
- **Unit:** mm
- **Compression:** Yes (compress: true)
- **Margin:** 0 (no white borders)
- **Image Source:** Canvas from html2canvas at scale 3

9.4 Print Specifications (Admin)

- **Size:** 320px width (centered on screen)
- **Fonts:** Tajawal (Google Fonts)
- **Colors:** `#3D0820` (maroon), `#c59b5f` (gold)
- **Image Loading:** JavaScript waits for all images before `window.print()`
- **@media print:** `page-break-inside: avoid` on all sections

9.5 Tracking Token System

Order Creation

```
|
|— phone → search customer_tokens
|   |— exists → use existing token (stable for customer)
|   |— not found → create new token (crypto.randomUUID)
|
|— Store token in orders.tracking_token
|
|— token ↔ QR Code on invoice
|
|— /t/[token] → order tracking page
|   |— Fetch order by tracking_token
|   |— Display status + details
|   |— InvoiceModal with QR
```

Section 10: Security & Authorization

10.1 Roles (3)

Role	Description	Permissions
developer	Full-access developer	Everything
manager	Restaurant manager	Dashboard, Menu, Categories, Offers, Gallery, Reviews, Reports, Settings, Delivery
order_manager	Order manager only	Orders only

10.2 Access Control

```
1. Middleware (src/middleware.ts)
  └─ Matches /admin/:path*
  └─ Calls updateSession from utils/supabase/middleware
  └─ Verifies:
    └─ Auth session exists
    └─ admin_users record exists
    └─ Page access permission (canAccessPage)

2. Server-Side (permissions.ts)
  └─ PAGE_ACCESS map
  └─ canAccessPage(role, pathname)
  └─ getAllowedSidebarLinks(role)
  └─ getDefaultRedirect(role)

3. RLS (Row Level Security)
  └─ rls.sql contains:
    └─ is_admin()      ← Check if admin in admin_users
    └─ get_admin_role() ← Get admin role
    └─ Public SELECT on: categories, items, item_prices, offers,
                        gallery_images, site_settings, branches
    └─ Admin-only INSERT/UPDATE/DELETE
    └─ Customers see only their orders (customer_phone)
    └─ Admins see all orders
```

10.3 Server Action Protection

- `createOrder` → `'use server'` → validates data only (not Auth)
- `updateOrderStatus` → `'use server'` → no direct Auth check (used by Admin)
- `calculateDeliveryFeeServer` → `'use server'` → no Auth (public service)
- Permission is enforced at middleware + sidebar level

10.4 Potential Vulnerabilities

Vulnerability	Status	Impact
<code>updateOrderStatus</code> without Auth check	⚠ Depends on Middleware	Low - callable from any Client
<code>createOrder</code> without CSRF protection	⚠ No CSRF Token	Medium
<code>calculateDeliveryFeeServer</code> public	✅ Intentional	Low - preview only
Tracking token guessable	✅ <code>crypto.randomUUID()</code>	Low
<code>site_settings</code> public read	✅ Intentional	Low

Section 11: Reporting System

11.1 Architecture

```
Report Interface (Client)
└─ /admin/reports/page.tsx
  └─ 9 Tabs
    ├── DashboardTab
    ├── OrdersTab
    ├── ProductsTab
    ├── CustomersTab
    ├── SummaryTab
    ├── OffersTab
    ├── InvoicesTab
    ├── AuditLogsTab
    └─ DeliveryAnalyticsTab
      └─ API Routes (12 endpoints)
        ├── REST (Supabase Client)
        └─ Drizzle ORM (server-side)
          └─ services/reports/
            ├── salesReport.ts
            ├── analyticsEngine.ts
            ├── customerReport.ts
            ├── dashboardReport.ts
            ├── ordersReport.ts
            └─ productReport.ts
```

11.2 Print & Export

Print (printReport.ts)

- └─ fetchData() → collect data from API
- └─ buildSection() → build complete HTML with inline CSS
- └─ Open print window (window.open)
- └─ wait + window.print()

Excel Export (exportExcel.ts)

- └─ ExcelJS → multi-sheet .xlsx
 - └─ Orders Sheet
 - └─ Products Sheet
 - └─ Customers Sheet
 - └─ Sales Comparison Sheet
 - └─ Delivery Analytics Sheet

Section 12: State Management (Stores)

12.1 cartStore (Zustand + persist)

```
// localStorage key: "bam-cart-storage"
// Middleware: persist

useCartStore(
  state: {
    items: CartItem[] // [id, name, price, quantity, size, category,
image, offer...]
  },
  actions: {
    addToCart(item) // Search for duplicate (id + size) → increment
quantity
    removeFromCart(id) // Remove item
    updateQuantity(id, quantity) // Change quantity (0 → remove)
    clearCart() // Empty cart
    getCartTotal() //  $\Sigma(\text{price} \times \text{quantity})$ 
    getTotalItems() //  $\Sigma(\text{quantity})$ 
  }
)
```

12.2 SettingsContext (React Context)

```
SettingsProvider
├─ refresh() → fetch site_settings from Supabase
├─ Transform data to Record<string, string>
└─ Provide { settings, loading, refresh } to children

useSettings() → { settings: Record<string, string>, loading: boolean }
```


Section 13: Legacy Files (Unused)

These files were part of the old zone-based pricing system and are no longer imported by any production code:

File	Reason
<code>src/lib/delivery-zones.ts</code>	Zone system (Azal, Shattar, etc.) — replaced by distance pricing
<code>src/lib/geo/resolve-zone.ts</code>	Zone resolution via Nominatim API — no longer used
<code>src/lib/geo/resolve-location.ts</code>	R-Tree spatial indexing engine — no longer used
<code>src/lib/geo/sanaa-boundaries.ts</code>	GeoJSON loading — no longer used
<code>src/app/api/resolve-zone/route.ts</code>	Zone resolution API — no longer used
<code>src/app/cart/page.tsx</code>	Old cart page — replaced by <code>/my-orders</code>
<code>src/app/track-order/[orderId]/page.tsx</code>	UUID-based tracking — replaced by <code>/t/[token]</code>
<code>delivery_zone</code> column in orders	Always NULL

Note: The `delivery_zones` table still exists in the database but is unused in any code. `delivery_zone: null` is set in `createOrder` for backward compatibility.

Section 14: Dependency Map



Section 15: Executive Summary

15.1 Strengths

- ☒ **Single Source of Truth** for delivery fees (`calculateDeliveryFee`)
- ☒ **All Calculations Server-Side** (client values never trusted)
- ☒ **Optimistic Locking** for orders (prevents concurrent updates)
- ☒ **Snapshots for offers and delivery** (ensures report accuracy)
- ☒ **4 Offer Types** (fixed, percentage, amount, free)
- ☒ **3-Way Tracking** (Admin UI, Token Link, QR Code)
- ☒ **3 Format Invoices** (PNG, PDF, Direct Print)
- ☒ **Complete Reporting System** (9 tabs + Print + Excel)
- ☒ **Extensible Database** (Drizzle ORM + SQL Migrations)
- ☒ **3 Roles** (Developer, Manager, Order Manager)

15.2 Recommended Improvements

Issue	Severity	Recommendation
<code>updateOrderStatus</code> without CSRF/Token	Medium	Add session verification inside Server Action
<code>createOrder</code> without Auth	Medium	Add Rate Limiting + additional validation
<code>fetchDeliverySettings</code> called twice (preview + creation)	Low	Merge the two calls or use caching
Legacy files not deleted	Low	Delete unused files (delivery-zones.ts, geo/*, cart/page.tsx)
Tracking Token persists forever	Low	Add token expiry
<code>services/reports/</code> uses Drizzle but some API routes use Supabase REST	Low	Unify approach (all Drizzle or all REST)
No DB Migration to drop <code>delivery_zones</code>	Low	Add migration to drop the table

15.3 Project Deliverables

File	Purpose
<code>BAIT_AL_MANDI_ARCHITECTURE.md</code>	Complete system documentation (Markdown)
<code>exports/BAIT_AL_MANDI_ARCHITECTURE_AR.pdf</code>	Arabic PDF documentation
<code>exports/BAIT_AL_MANDI_ARCHITECTURE_EN.pdf</code>	English PDF documentation